

Package ‘BPSMKL’

June 29, 2026

Title Bayesian Pathway-Structured Multiple Kernel Kearning (BPS-MKL)

Version 0.0.0.9000

Description BPSMKL is an R package for simultaneously identifying important groups, features within groups, and interactions among features.

License GPL (>= 3)

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2.9000

URL <https://github.com/Yuzi896/BPSMKL>

BugReports <https://github.com/Yuzi896/BPSMKL/issues>

LinkingTo Rcpp,
RcppArmadillo

Imports Rcpp

Suggests ggplot2,
patchwork,
pROC

R topics documented:

BPSMKL-package	2
bps_mkl	2
bps_mkl_app	4
get_K	6
res_summary	7
simulate_pathway	8
Index	10

BPSMKL-package

BPSMKL package

Description

Bayesian pathway sparse multiple kernel learning.

Author(s)

Maintainer: First Last <first.last@example.com>

See Also

Useful links:

- <https://github.com/Yuzi896/BPSMKL>
- Report bugs at <https://github.com/Yuzi896/BPSMKL/issues>

bps_mkl

Run BPS-MKL from individual initial values

Description

bps_mkl() collects the individual initial values into the list expected by the C++ sampler and then runs BPS-MKL algorithm.

Usage

```
bps_mkl(
  iterations,
  burn,
  L,
  As,
  Y,
  X,
  P_eta = 0.2,
  P_gamma = 0.2,
  sigma2 = 0.1,
  lambdas = NULL,
  betas = NULL,
  gammas = NULL,
  Etas = NULL
)
```

Arguments

<code>iterations</code>	Integer. Total number of MCMC iterations to run.
<code>burn</code>	Integer. Number of burn-in iterations used for proposal adaptation.
<code>L</code>	Integer. Number of pathways.
<code>As</code>	List of L numeric adjacency matrices. Each matrix defines the candidate main effects and interactions for one pathway.
<code>Y</code>	Numeric response vector or $n \times 1$ matrix.
<code>X</code>	Numeric $n \times p$ design matrix, with observations in rows and predictors in columns.
<code>P_eta</code>	Numeric scalar. eta inclusion probability.
<code>P_gamma</code>	Numeric scalar. gamma inclusion probability.
<code>sigma2</code>	Numeric scalar. initial residual variance.
<code>lambdas</code>	Numeric vector of length $2 * L$, with interaction and main-effect bandwidths for each pathway.
<code>betas</code>	Numeric vector of length L containing initial beta values.
<code>gammas</code>	Numeric vector of length L containing initial gamma values.
<code>Etas</code>	List of L numeric eta matrices.

Value

A list with MCMC draws and diagnostics:

`gamma_save` An iterations by L matrix of gamma draws.
`beta_save` An iterations by L matrix of beta draws.
`sigma_save` A numeric vector of `sigma2` draws.
`eta_whole` A list of L matrices storing eta draws for the candidate terms in each pathway.
`eta_idx` A list of L two-column matrices giving the one-based predictor indices corresponding to the columns of `eta_whole`.
`lambda_save` An iterations by $2 * L$ matrix of lambda draws.
`ll_save` A numeric vector of log-likelihood values.
`time` Time used to run the algorithm, returned as a `difftime` object.

Examples

```
## Not run:
set.seed(1)
L <- 3
n <- 100
p <- 50
sigma2 <- 1
beta <- c(0, 2, 2)
gamma <- ifelse(beta == 0, 0, 1)

pathway <- simulate_pathway(
  p = p,
  n_pathways = L,
  pathway_p = 10,
  n_edges = 15,
```

```

    n_imp_edges = 5,
    n_imp_main = 2,
    main = TRUE,
    seed = 1
  )

X <- matrix(runif(n * p), nrow = n, ncol = p)
X <- scale(X)

A <- pathway$A
A_imp <- pathway$A_imp
FX <- matrix(0, nrow = n, ncol = L)

for (l in seq_len(L)) {
  K <- get_K(X, A_imp[[l]], c(1, 1))
  # Sample fx from MVN(0, K).
  FX[, l] <- drop(t(chol(K + 1e-8 * diag(n))) %*% rnorm(n))
}

Y <- drop(FX %*% beta + rnorm(n, sd = sqrt(sigma2)))

result <- bps_mkl(
  iterations = 5000,
  burn = 1000,
  L = L,
  As = A,
  Y = Y,
  X = X,
  sigma2 = sigma2,
  betas = beta,
  gammas = gamma,
  Etas = A_imp
)

## End(Not run)

```

bps_mkl_app

Run approximate BPS-MKL from individual initial values

Description

bps_mkl_app() collects the individual initial values into the list expected by the optimized C++ sampler and then runs the approximate Nyström BPS-MKL algorithm.

Usage

```

bps_mkl_app(
  iterations,
  burn,
  L,
  As,
  Y,
  X,

```

```

    m,
    P_eta = 0.2,
    P_gamma = 0.2,
    sigma2 = 0.1,
    lambdas = NULL,
    betas = NULL,
    gammas = NULL,
    Etas = NULL
)

```

Arguments

iterations	Integer. Total number of MCMC iterations to run.
burn	Integer. Number of burn-in iterations used for proposal adaptation.
L	Integer. Number of pathways.
As	List of L numeric adjacency matrices. Each matrix defines the candidate main effects and interactions for one pathway.
Y	Numeric response vector or $n \times 1$ matrix.
X	Numeric $n \times p$ design matrix, with observations in rows and predictors in columns.
m	Integer. Number of observations, or rows of X, to sample as Nyström landmarks.
P_eta	Numeric scalar. eta inclusion probability.
P_gamma	Numeric scalar. gamma inclusion probability.
sigma2	Numeric scalar. initial residual variance.
lambdas	Numeric vector of length $2 * L$, with interaction and main-effect bandwidths for each pathway.
betas	Numeric vector of length L containing initial beta values.
gammas	Numeric vector of length L containing initial gamma values.
Etas	List of L numeric eta matrices.

Value

A list with MCMC draws and diagnostics:

gamma_save An iterations by L matrix of gamma draws.

beta_save An iterations by L matrix of beta draws.

sigma_save A numeric vector of sigma2 draws.

eta_whole A list of L matrices storing eta draws for the candidate terms in each pathway.

eta_idx A list of L two-column matrices giving the one-based predictor indices corresponding to the columns of eta_whole.

lambda_save An iterations by $2 * L$ matrix of lambda draws.

ll_save A numeric vector of log-likelihood values.

time Time used to run the algorithm, returned as a difftime object.

Examples

```
## Not run:
# Assume simulation objects are already pre-generated.
# See details in the `bps_mkl()` example.
result <- bps_mkl_app(
  iterations = N,
  burn = burnin,
  L = L,
  As = A,
  Y = Y,
  X = X,
  m = 50,
  sigma2 = sigma2,
  betas = beta,
  gammas = gamma,
  Etas = A_imp
)

## End(Not run)
```

get_K

Construct a dense kernel matrix for selected effects

Description

get_K() builds the N by N kernel matrix used by the model from an input design matrix, an effect-selection matrix, and two bandwidth parameters. The candidate main-effect and interaction index matrix is computed internally from Eta.

Usage

```
get_K(X, Eta, lambda, update = FALSE, eta = NULL, i = NULL, j = NULL)
```

Arguments

X	Numeric matrix with observations in rows and predictors in columns.
Eta	Numeric square matrix indicating which main effects and interactions are active. Diagonal entries represent main effects and off-diagonal entries represent interactions.
lambda	Numeric vector of length 2. lambda[1] is the bandwidth for interaction terms and lambda[2] is the bandwidth for main effects.
update	Logical flag. Use FALSE to compute the kernel from Eta as supplied, or TRUE to compute it after temporarily updating the (i, j) and (j, i) entries of Eta.
eta	Proposed value to use when update = TRUE; must be 0 or 1. Defaults to NULL.
i	One-based row index in Eta to update when update = TRUE. Defaults to NULL.
j	One-based column index in Eta to update when update = TRUE. Defaults to NULL.

Details

Off-diagonal active entries in `Eta` are treated as interactions and use `lambda[1]`; diagonal active entries are treated as main effects and use `lambda[2]`. When `update = TRUE`, the C++ helper works with a temporary copy of `Eta` where `Eta[i, j]` and `Eta[j, i]` are set to `eta`. The candidate index matrix is always computed from the original `Eta`.

Value

An N by N numeric kernel matrix, where $N = \text{nrow}(X)$.

<code>res_summary</code>	<i>Summarize BPS-MKL MCMC results</i>
--------------------------	---------------------------------------

Description

`res_summary()` summarizes posterior pathway and eta inclusion probabilities from a fitted BPS-MKL result object. It always returns an `idx_res` data frame with pathway labels, predictor index pairs, and estimated eta probabilities. When simulated truth is supplied through `pathway`, it also returns pathway-level AUC values. When `Figure = TRUE` and `ggplot2` is available, it also returns diagnostic heatmaps.

Usage

```
res_summary(
  res,
  pathway = NULL,
  gammas = NULL,
  p = NULL,
  burnin = 0,
  Figure = FALSE
)
```

Arguments

<code>res</code>	Result list returned by <code>bps_mkl()</code> or <code>bps_mkl_app()</code> .
<code>pathway</code>	Optional simulated pathway object, such as the output from <code>simulate_pathway()</code> . When supplied, <code>pathway\$A_imp</code> is used as truth for AUC calculation and optional figures.
<code>gammas</code>	Optional numeric vector of true pathway inclusion indicators. When supplied with <code>pathway</code> and <code>Figure = TRUE</code> , truth heatmaps multiply <code>A_imp</code> by <code>gammas</code> .
<code>p</code>	Optional integer number of predictors. If <code>NULL</code> , this is inferred from <code>dim(res\$eta_whole[[1]])[1]</code> .
<code>burnin</code>	Integer number of initial MCMC draws to discard. Defaults to 0.
<code>Figure</code>	Logical. If <code>TRUE</code> , return heatmap figures when <code>ggplot2</code> is available. Defaults to <code>FALSE</code> .

Value

A list containing:

`idx_res` A data frame with `pathway_id`, `idx1`, `idx2`, `estimated`, and `P_pathway`. If `pathway` is supplied, it also contains `truth`.

`auc_sim` A data frame with pathway-level AUC values. Returned only when `pathway` is supplied.

`Figure` A list of pathway heatmaps. Returned only when `Figure = TRUE` and `ggplot2` is available. If `pathway` is supplied, `truth` heatmaps are included.

<code>simulate_pathway</code>	<i>Simulate pathway adjacency matrices</i>
-------------------------------	--

Description

`simulate_pathway()` creates non-overlapping simulated pathways over `p` features. For each pathway, it samples `pathway_p` features, creates `n_edges` candidate interaction edges among those features, and marks `n_imp_edges` of those candidate edges as important. Optionally, it can also add important main effects on the diagonal of each important-effect matrix.

Usage

```
simulate_pathway(
  p,
  n_pathways,
  pathway_p,
  n_edges,
  n_imp_edges,
  n_imp_main = 0,
  main = FALSE,
  seed = NULL
)
```

Arguments

<code>p</code>	Integer. Total number of features.
<code>n_pathways</code>	Integer. Number of pathways to simulate.
<code>pathway_p</code>	Integer. Number of features assigned to each pathway.
<code>n_edges</code>	Integer. Number of candidate interaction edges to include in each pathway adjacency matrix.
<code>n_imp_edges</code>	Integer. Number of candidate interaction edges to mark as important in each pathway.
<code>n_imp_main</code>	Integer. Number of pathway features to mark as important main effects when <code>main = TRUE</code> . Defaults to 0.
<code>main</code>	Logical. If <code>TRUE</code> , add important main effects on the diagonal of each important-effect matrix. Defaults to <code>FALSE</code> .
<code>seed</code>	Optional integer random seed used before simulation. Defaults to <code>NULL</code> .

Value

A list with two elements:

A A list of $n_pathways$ $p \times p$ adjacency matrices containing the candidate interaction edges for each pathway.

A_imp A list of $n_pathways$ $p \times p$ matrices containing the important interaction edges and, when requested, important main effects.

Index

`bps_mkl`, [2](#)
`bps_mkl_app`, [4](#)
BPSMKL (BPSMKL-package), [2](#)
BPSMKL-package, [2](#)

`get_K`, [6](#)

`res_summary`, [7](#)

`simulate_pathway`, [8](#)